

System Architecture

Purpose: Describes the high-level structure of Minesweep implementation to facilitate feature extensions by Project 2 Team

Components:

- **Frontend (JS/HTML):** Turns the 10x10 board into visible user output. Takes in user input in the form of clicks & sends corresponding calls to the API.
- **Middle Man (fastAPI):** Based on API calls from the frontend, directs input to the Minesweeper class and returns the resulting game state (board, state, remaining flags).
- **Backend/Game Logic (python):** This is handled within [Minesweeper.py](#). This class takes user input and handles two 10x10 grids to track the full user display. One grid holds a visible copy that is passed directly up. The other holds the authoritative copy of where the mines are. This class handles all the game logic including mine placement and uncovering cells.

Data Flow:

- User Input (click) -> Frontend -> Corresponding API call
- API calls -> FastAPI -> Backend
- Backend then update based on the call and creates a new board state
- New board state -> FastAPI -> Frontend -> Displayed to user

Key Data Structures:

- Minesweeper class: This stores all the relevant information about the game (includes game state, flag count, and matrices listed below) & handles all game logic.
 - Visited Array: This represents the visible part of a 10x10 matrix or 2D array which is then displayed to the user
 - Matrix array: This is a 10x10 matrix or 2D array which is the authoritative source of where the mines are in the minesweeper game
- Major Variable Definitions for matrix/board state
 - 0 = Empty square
 - -1 = Flagged
 - -2 = Exploded Mine
 - -3 = Mine
 - Unvisited = NONE
 - Visited = Number > 0
 - Number represents adjacent mines
- Game States
 - 1 = Win
 - 2 = Running / ongoing
 - 0 = Loss

Assumptions:

- Static 10x10 grid

- Mine count will be specified by user at the start